



Dig the diff, parse the past: Eine Zeitreise durch Git-Repositories

Christian Budde [@MeKo-Christian](https://github.com/MeKo-Christian)



<https://github.com/MeKo-Christian/hercules>

Worum geht es?

Git-History als forensisches Artefakt

Jeder Commit ist ein unveränderlicher Snapshot. Tausende Commits. Jahre an Code-Changes. Ein kompletter Audit-Trail, der nur darauf wartet, geparkt zu werden.

Was wir uns anschauen:

-  Code-Archäologie: Wie hat sich unsere Codebasis entwickelt?
-  Statistische Patterns: Hotspots, Coupling, Ownership-Distribution
-  Team-Dynamiken quantifizieren: Commit-Frequenzen, Co-Change-Patterns
-  Historie als Datenquelle: Was können wir aus dem Commit-Graph extrahieren?

Warum Git-History Analytics?

Git vergisst nie – Perfekt für Metriken! 🚩

Immutable History. Ein kompletter Audit-Trail. Wir machen Data Mining im .git-Verzeichnis!

Traditionelle Metriken haben ihre Grenzen:

- Anzahl Commits $\neq$ Produktivität
- Lines of Code $\neq$ Wertbeitrag
- “Wer war aktiv?” $\neq$ “Wer versteht das System?”

Git-Historie kann mehr zeigen →

Was Git-Historie zeigen kann



Evolution

- Wachstumsphasen
- Refactoring-Wellen
- Architektur-Drifts



Risiken

- Code-Hotspots
- Wissenskonzentration
- Bus-Factor-Gefährdung



Ownership

- Wer besitzt was?
- Wie verteilt sich Wissen?
- Onboarding-Muster



Collaboration

- Co-Change-Muster
- Implizite Teams
- Zusammenarbeitssignale

Was ist Hercules?

Git-History Parser → Metrics → Visualisierung

Git Repo → hercules (Go) → labours (Python) → Charts

└─ YAML/PB	└─ Matplotlib
└─ Analysen	└─ Plots

Composable Pipeline:

- Single-pass über Commits, parallele Aggregation
- Erweiterbar ohne Core-Änderungen

Was Hercules sieht (und was nicht)

✓ Was Hercules analysiert:

- Commits (Autor, Datum, Message)
- Diffs (Added/Removed/Changed Lines)
- File-Renames & Moves
- Branches & Merges
- Code-Struktur (via Babelfish/UAST)
- Kommentare (optional: Sentiment)

Quelle: Git-Objekt-Datenbank

✗ Was Hercules NICHT sieht:

- Pull Requests & Reviews
- Tickets & Issues
- CI/CD-Ergebnisse
- Deployment-Historie
- Team-Kommunikation
- Code-Qualitätsmetriken (Test Coverage, etc.)

⚠ Wichtig für Interpretation!

Typischer Workflow

Schritt 1: Metrics Extraction

```
hercules --burndown --couples --devs \  
https://github.com/user/repo > output.yaml
```

Schritt 2: Visualization Pipeline

```
labours -m burndown-project -f yaml \  
output.yaml -o burndown.png
```

Serialization Formats:

- **YAML:** Human-readable, aber $O(n^2)$ parse-time bei großen Repos
- **Protocol Buffers** (`--pb`): Binary, optimiert für große Datasets
- **JSON:** Via `labours` für Custom Viz oder Weiterverarbeitung

Metrik-Überblick



Evolution & Alter

- Project Burndown
- File Burndown
- People Burndown



Hotspots & Risiken

- Structural Hotness (Shotness)
- File Churn
- Coupling-basierte Hotspots



Team & Ownership

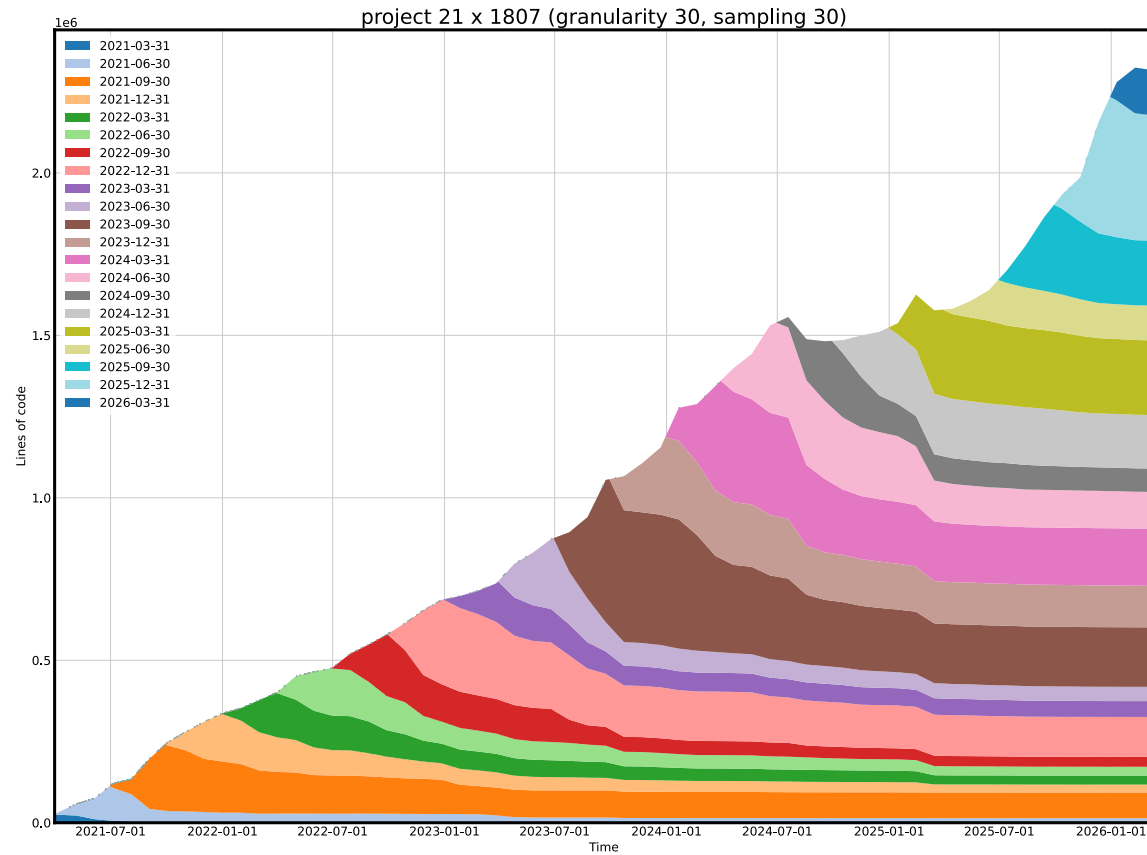
- Code Ownership über Zeit
- Overwrites Matrix (Churn A→B)
- Developer Coupling



Zeitreihen & Aktivität

- Aligned Commit Series
- Added vs. Changed Lines
- Efforts Through Time
- Sentiment (optional)

Project Burndown: Code-Alter-Verteilung



Project Burndown: Interpretation

Was ist Burndown?

- Zeigt, wie viele *heute lebende* Zeilen aus welchen Zeitbändern stammen
- Inkrementelles Blaming ($O(n)$ statt $O(n^2)$)
- “Last modification date” pro Zeile wird gespeichert

Wie lesen wir das?

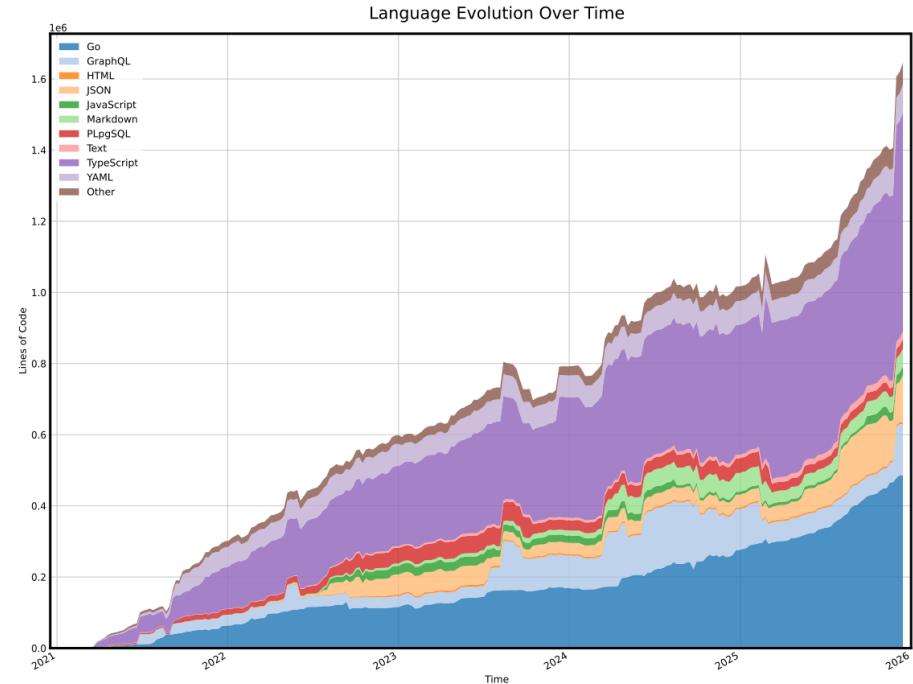
- Stabiler “alter” Anteil → langlebige Kernkomponenten
- Plötzlicher Shift zu “jung” → Rewrite/Refactoring-Welle
- Kontinuierliches Wachstum → Feature-Expansion

Sprachen-Verteilung: Woraus besteht die Codebase?

Language Statistics über Zeit

Zeigt Entwicklung der
Programmiersprachen:

-  Lines of Code pro Sprache
-  Wachstum einzelner Sprachen
-  Technologie-Shifts erkennbar



Sprachen-Verteilung: Interpretation

Typische Patterns:

- Polyglotte Repos: Mehrere Sprachen gleichzeitig
- Migrationen: Z.B. JavaScript → TypeScript
- Neue Technologien: Wachsende neue Sprachen
- Config & Tooling: YAML, JSON, etc.

Was sagt uns das?

-  Tech-Stack Evolution sichtbar
-  Komplexität durch Sprach-Diversität
-  Mehr Sprachen = mehr Kontext-Switches für Devs

Files & Hotspots: Risikoerkennung

Structural Hotness (“Shotness”)

- Anzahl Modifikationen je struktureller Einheit (z.B. Funktionen)
- Nutzt Babelfish/UAST für strukturelles Parsing
- Anpassbar via XPath (z.B. Klassen statt Funktionen)

Was bedeuten Hotspots?

- 🔥 Hot Functions/Classes = werden häufig geändert
- Mögliche Ursachen: Volatile Requirements, schlechte Abstraktion, komplexe Kernlogik

Risiko-Indikatoren (kombiniert):

`Risk Score = log(size) × churn × coupling × ownership_concentration`

People & Contributions

People Burndown: Wer schreibt was?

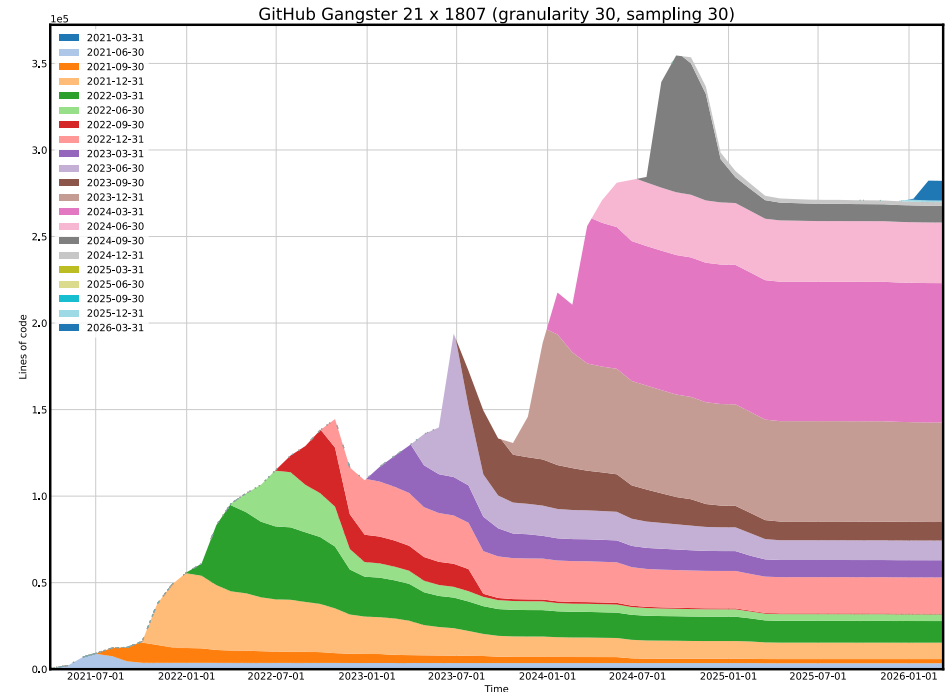
Definition:

- Burndown pro Contributor
- Identity-Resolution: --people-dict oder heuristisch
- Nutzt .mailmap (falls vorhanden)

Visualisierung: Stack-Area über Zeit

-  Lebende Zeilen pro Person
-  Zeitliche Entwicklung
-  Wachstum, Stabilität, Abnahme

Rechts: Beispiel eines intensiven Contributors →



People Burndown: Interpretation

Was sehen wir?

- Onboarding-Phase (steiler Anstieg)
- Aktive Maintenance (Plateau)
- Teamwechsel, Abgänge erkennbar
- Exit oder Projektwechsel

Interpretation (Vorsicht geboten!):

-  Ownership-Zeitreihe, “Onboarding Ramp” sichtbar
-  Teamwechsel und Abgänge erkennbar
-  Aber NICHT: “Produktivität” oder “Wertbeitrag”
-  Wird verzerrt durch: Pairing, Squash-Merges, Code-Gen

 **Line ownership $\neq$ Domain-Wissen oder tatsächlicher Beitrag!**

Overwrites Matrix: Wer überschreibt wen?



Overwrites Matrix: Interpretation

Was ist die Overwrites Matrix?

- Zeigt, wie viel Code von Person A durch Person B überschrieben wird
- “Churn A→B”: Zeilen, die A geschrieben hat und B später ändert
- Matrix-Darstellung: Reihe = Original-Autor, Spalte = Überschreiber

Was sehen wir?

-  Code-Review-Patterns: Wer überarbeitet wessen Code?
-  Mentoring-Beziehungen: Erfahrene überarbeiten Code von Juniors
-  Refactoring-Wellen: Systematische Code-Überarbeitungen
-  Maintenance: Wer pflegt welche Code-Bereiche nach?

Interpretation (Vorsicht!):

-  Zeigt Zusammenarbeitsmuster und Code-Pflege

-  Kann auf Wissensdiffusion hinweisen
-  Aber: Overwrites $\neq$ “schlechter Code” oder “Konflikte”
-  Normale Refactorings und Feature-Erweiterungen sind erwünscht!

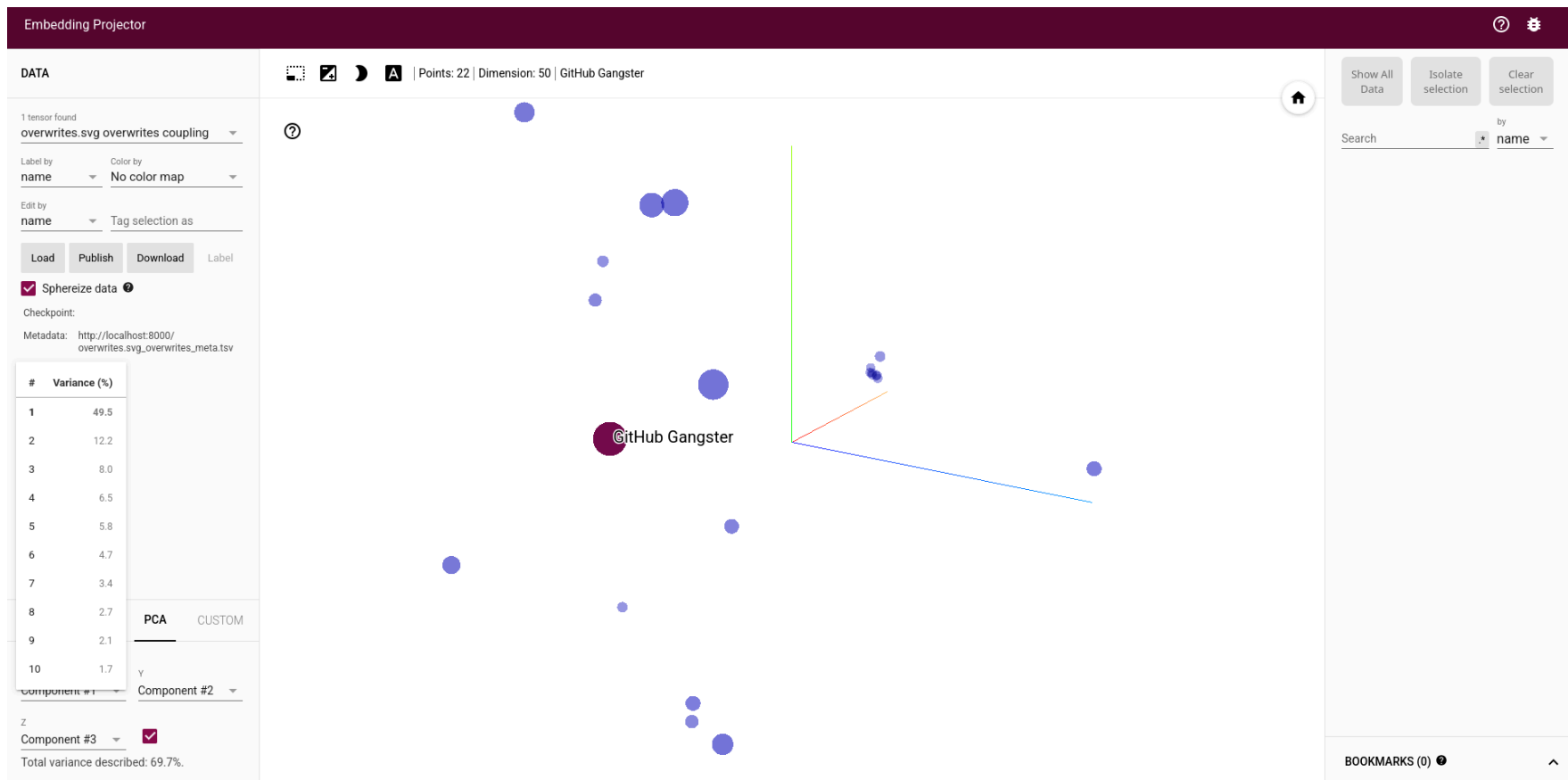
Couples: Zusammenarbeitssignale

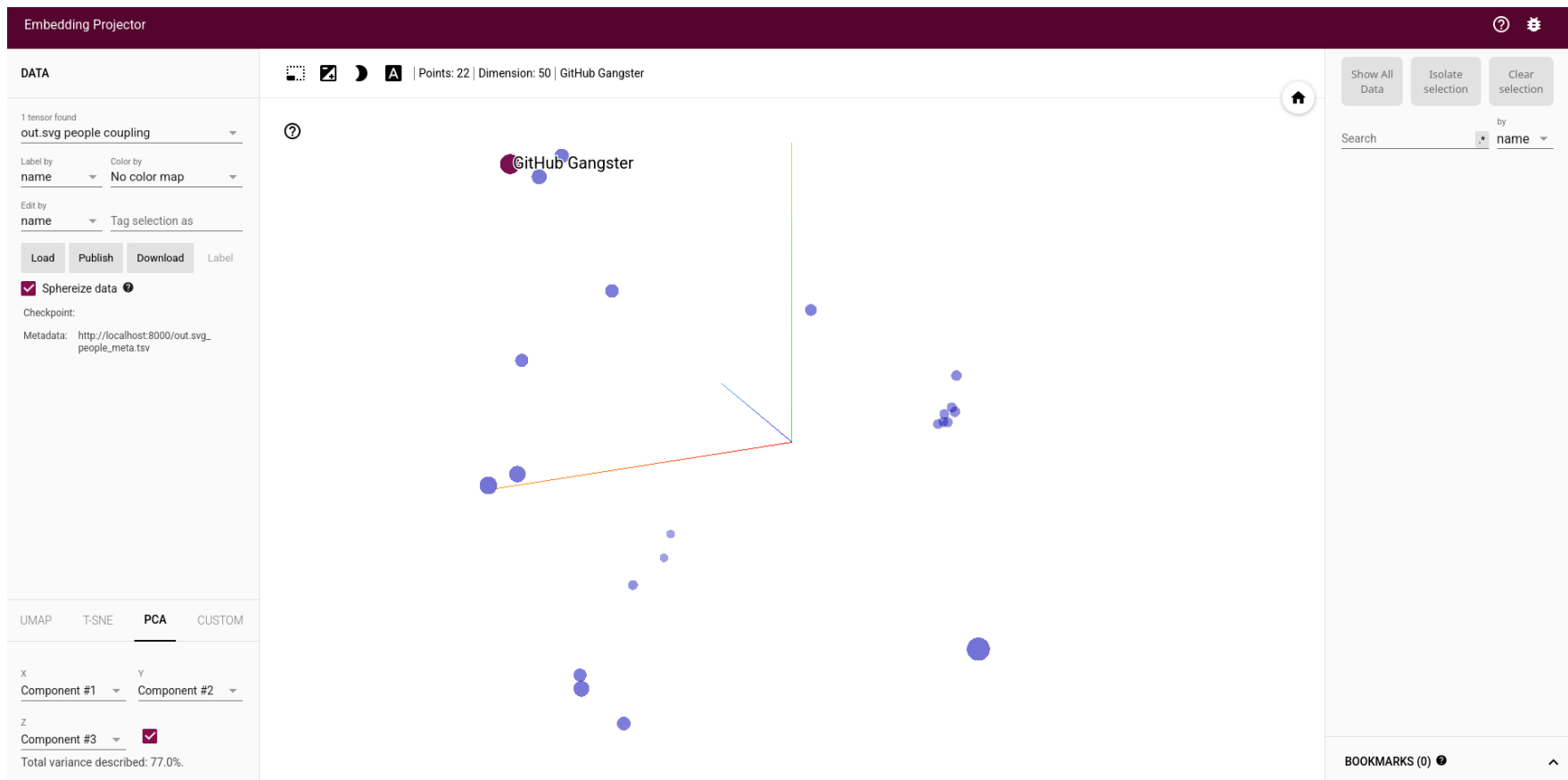
Was ist Coupling?

- **File-Coupling:** Dateien im gleichen Commit geändert
- **Developer-Coupling:** Entwickler ändern gleiche Dateien

Methodik:

1. Co-Occurrence-Matrix sammeln über Historie
2. Swivel Embeddings lernen ($N \times N \rightarrow N \times 100$)
3. t-SNE für 3D-Visualisierung (Tensorflow Projector)
4. “More common commits $\Rightarrow$ closer in 3D”





Couples: Interpretation

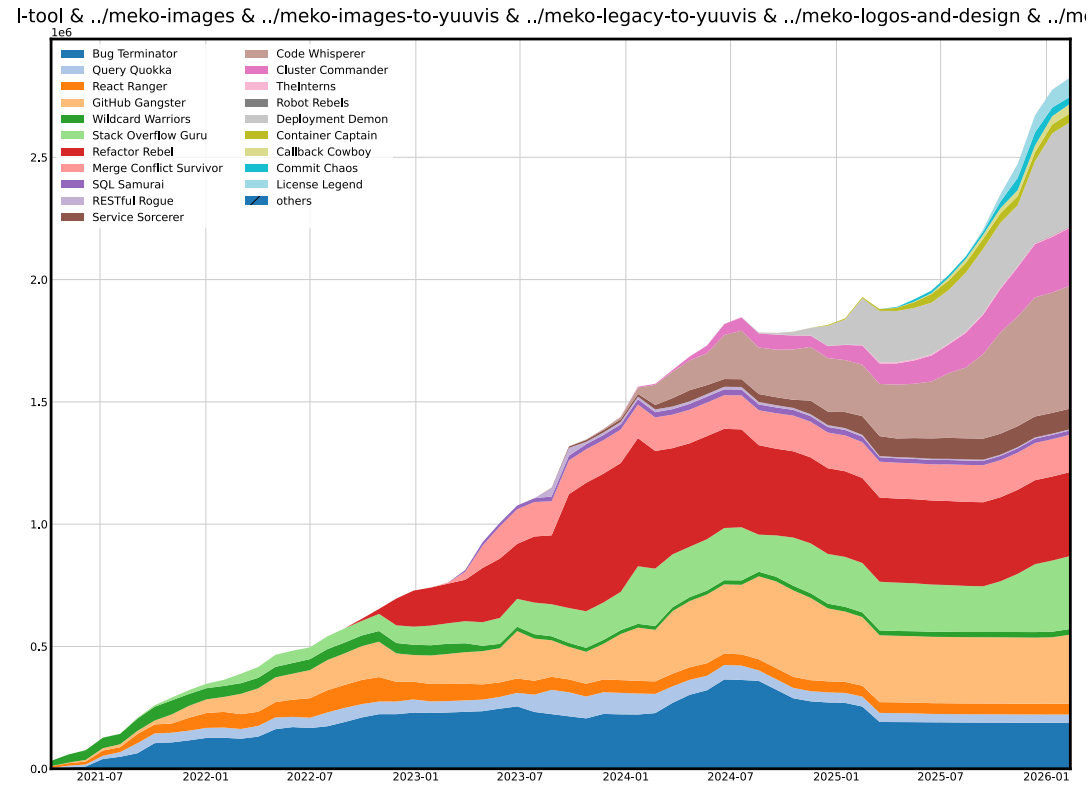
Was sehen wir damit?

-  Architektur: Modulare Cluster oder ein Monolith?
-  Implizite Subteams: Wer arbeitet wo zusammen?
-  Kopplungszonen: Welche Files ändern sich immer gemeinsam?

Wichtig zu wissen:

- Coupling $\neq$ “Kommunikation” (es ist nur ein Proxy!)
- Kann durch Code-Owner-Modell oder Monorepo-Struktur entstehen

Code Ownership über Zeit



Ownership & Bus-Factor: Interpretation

Code Ownership = “Code Share Through Time”

- Stack-Area: Lebende Zeilen pro Person über Zeit
- Basiert auf People Burndown Snapshotting

Bus-Factor-Proxy (nicht explizit im Tool):

Bus-Factor@80% = kleinstes k , sodass
 $\text{Sum}(\text{Top-}k \text{ Owners}) \geq 80\% \text{ aller lebenden Zeilen}$

Wie interpretieren wir das?

- **Konzentration** (wenige Personen, hoher Anteil):
 -  Wissenskonzentration, Bus-Factor-Risiko
 -  Aber: kann auch gute Spezialisierung sein

- **Diversifizierung** (viele Personen, stabile Anteile):
 -  Breites Wissen, resilientes Team
 -  Aber: kann auch fehlende Ownership bedeuten

 **“Ownership” hier = Zeilenbesitz, nicht zwingend Domain-Wissen!**

Zeitreihen-Ansichten

Aligned Commit Series: Team-Telemetrie

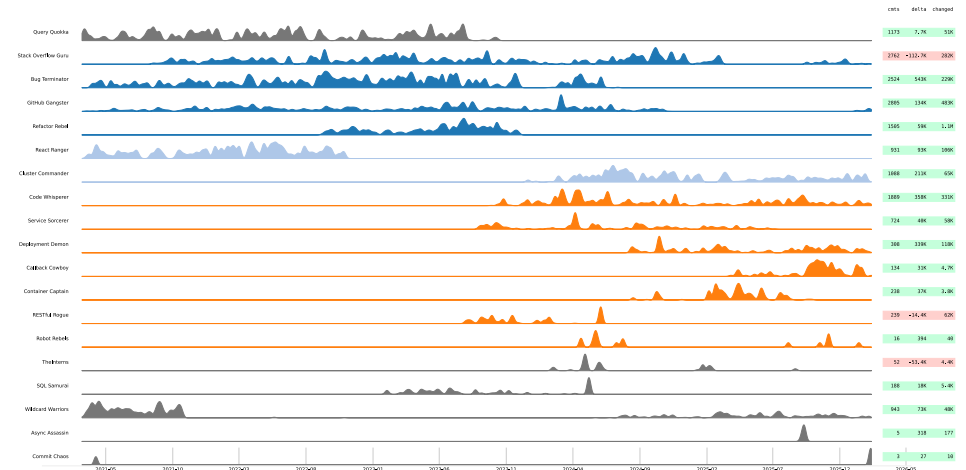
--devs erzeugt pro Entwickler:

- Commits pro Tag
- Lines added/removed/changed pro Tag

Visualisierung (4-Schritte-Prozess):

1. Normalisierung der Zeitreihen
2. DTW-Distanzmatrix (Dynamic Time Warping)
3. Seriation/TSP für Anordnung
4. HDBSCAN für Cluster-Farben + Slepian-Glättung

Rechts: Zeitreihen-Heatmap mit Clustering →



Aligned Commit Series: Interpretation

Was sehen wir hier?

-  Aktivitätswellen (z.B. “Hacktoberfest Flashmobs”)
-  Wer kommt, wer geht
-  Gruppen ähnlicher Aktivitätsmuster (z.B. Maintainer-Kern)

Wichtig zu wissen:

- Commits/Tag stark abhängig vom Workflow (Squash, Rebase, Pairing)
- Interpretation braucht immer Kontext!

 **Commit-Frequenz $\neq$ Produktivität oder Wertbeitrag!**

Added vs. Changed Lines

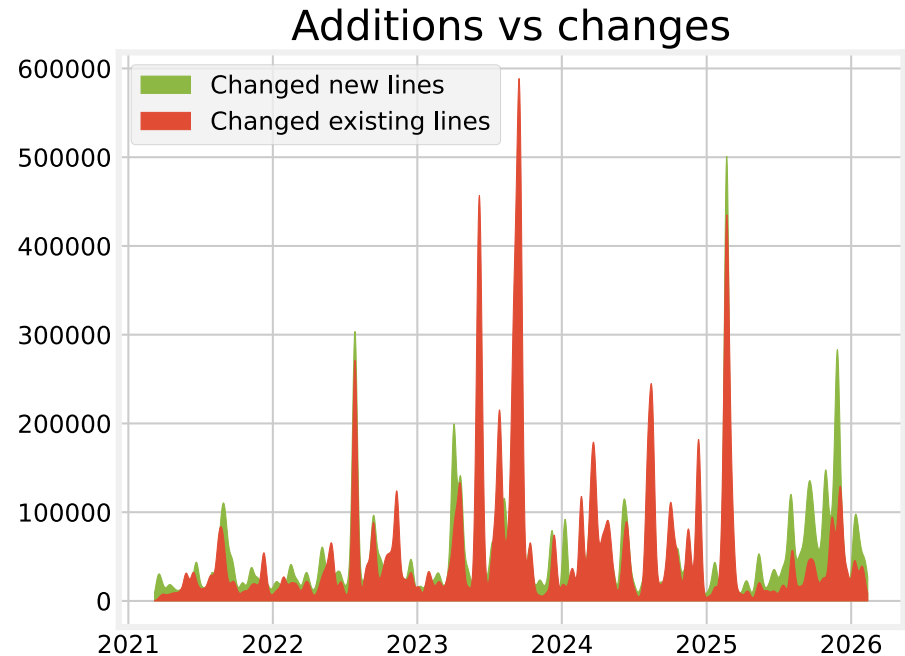
Zeitreihe zeigt:

-  Neu hinzugefügte Zeilen
-  Veränderte Zeilen (gelöscht/ersetzt)

Was heißt das?

- Viel “Added” → Feature-Expansion
- Viel “Changed” → Stabilisierung, Refactoring, Bugfixing

Rechts: Added vs. Changed Lines über Zeit →



Efforts Through Time

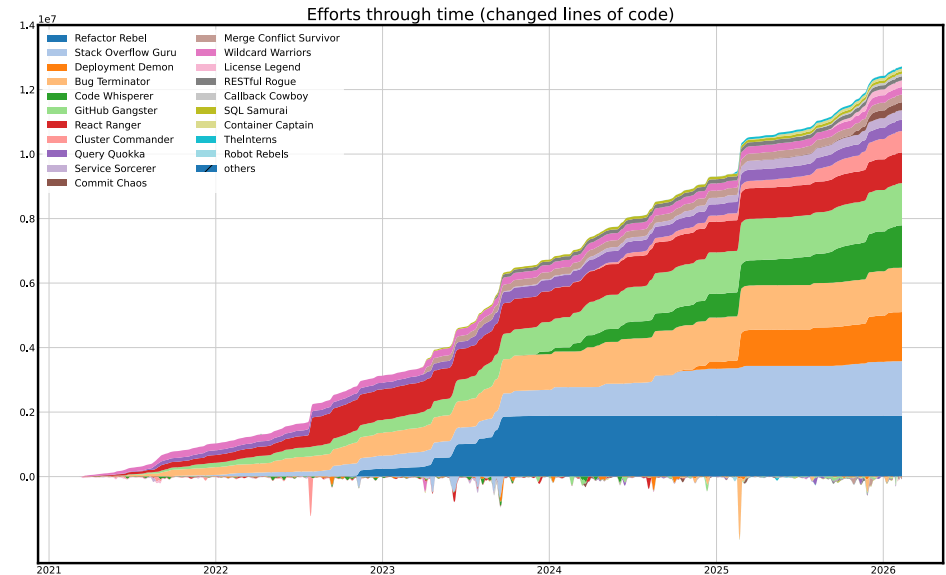
Pro Dev:

- Lines changed (added/removed)
- Obere Fläche = kumuliert

Was bedeutet das?

- Aktivitätsverlagerung zwischen Personen
- “Staffing Events” werden sichtbar
- ⚠ Aber: Effort ≠ Wertbeitrag!

Rechts: Stacked Area Chart pro Developer →



Commit-Zeiten: Wann wird entwickelt?

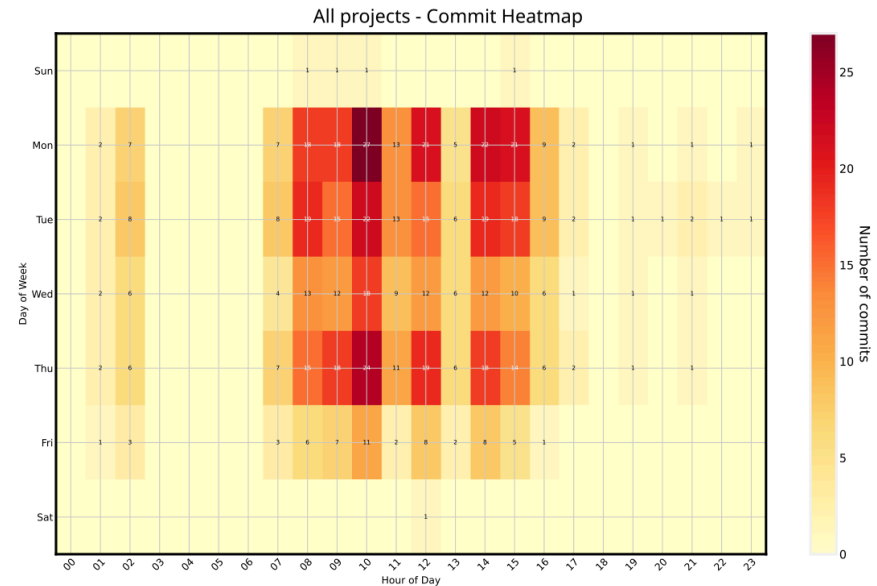
Aktivitätsmuster im Tagesverlauf

Heatmap: Commit-Zeiten aggregiert über:

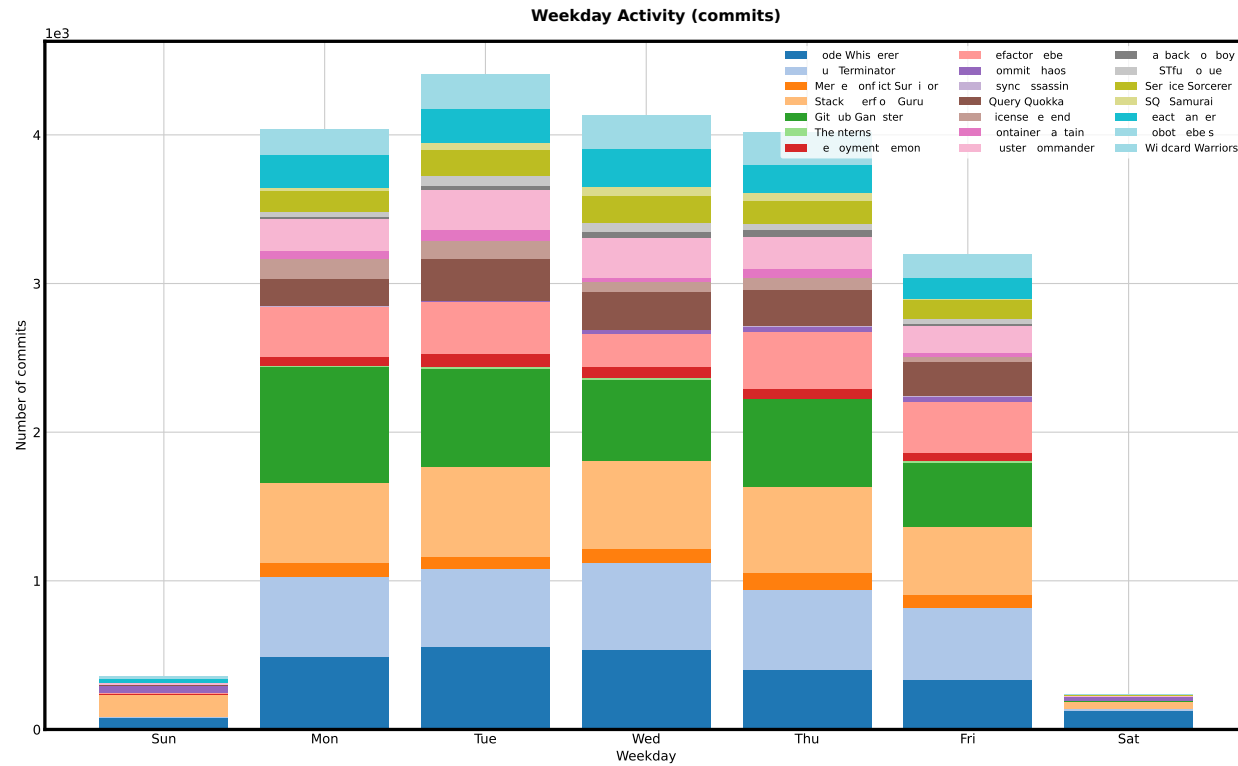
- 📅 Wochentag (Montag bis Sonntag)
- 🕒 Uhrzeit (0-23 Uhr)

Was zeigt uns das?

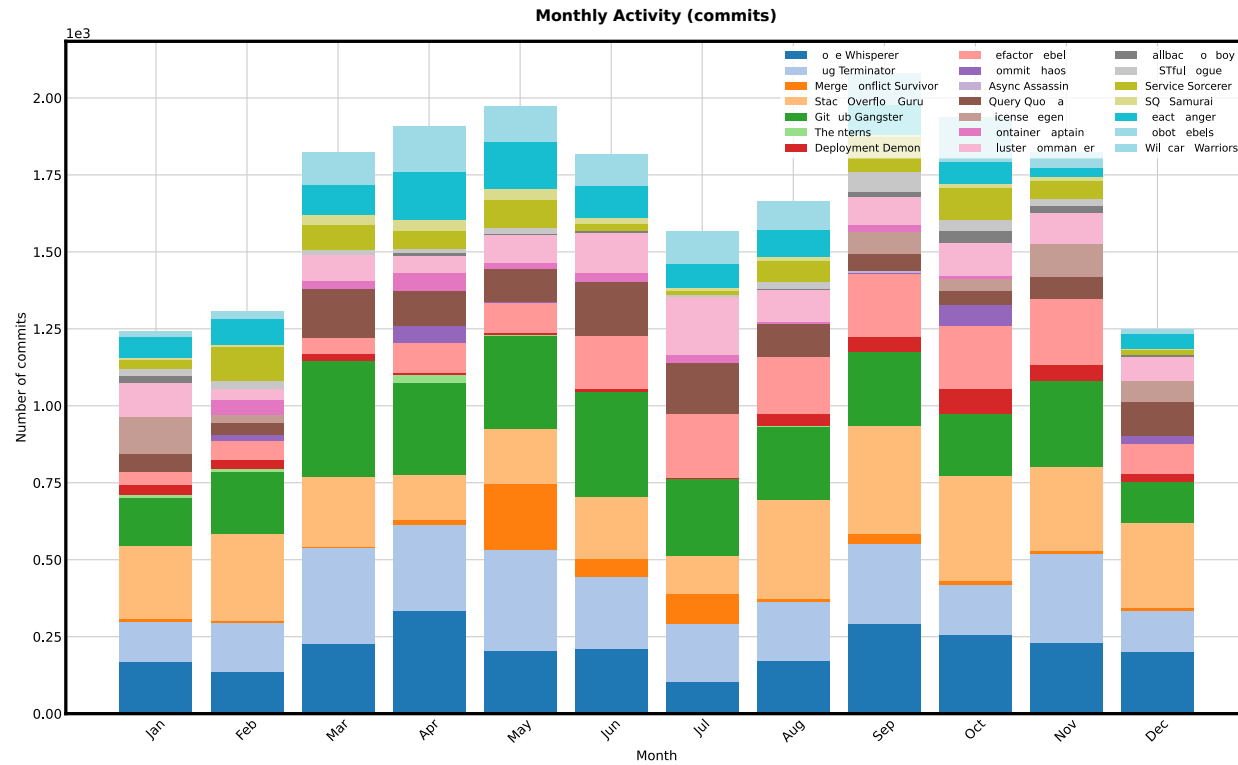
- 🕒 Arbeitsrhythmen: Core hours vs. flexible Zeiten
- 📊 Kulturelle Muster: Wochenendarbeit, Spätschichten



Aktivität nach Wochentagen



Aktivität nach Monaten



Commit-Zeiten: Interpretation

Interpretation (mit Vorsicht):

✓ Nützliche Einblicke:

- Team-Rhythmus verstehen, Meeting-Zeiten optimieren
- Burnout-Signale? (z.B. konstante Nachtschichten)
- Verteilte Teams & Zeitzone-Überlappungen identifizieren

✗ Wichtige Einschränkungen:

- Commit-Zeit $\neq$ Arbeitszeit
- Rebases und verzögerte Pushes verfälschen Daten
- Workflow-abhängig (Squash-Merges, CI-Commits)
- Keine Aussage über Produktivität oder Qualität

Sentiment: Experimentelles Feature

Was ist es?

- Analysiert neue/geänderte Kommentare pro Commit
- BiDiSentiment-Modell (RNN) via Tensorflow
- Output: Positiv/Negativ-Scores über Zeit

Voraussetzungen:

- Build mit Tag `tensorflow`
- `libtensorflow` erforderlich

Wichtige Einschränkungen:

README sagt: "don't expect too much"

- Modell ist general purpose, Code-Kommentare sind anders
- Ironie, Fachsprache, "negative" Wörter in neutralen Kontexten

Wie können wir das nutzen?

-  Als Diskussionsimpuls: “Wie sprechen wir im Code?”
-  Aber bitte NICHT als KPI verwenden!
-  Keine Kultur-/Stress-Diagnose ohne Kontext möglich



Grenzen & Fallstricke

Was wir NICHT tun sollten

🚫 Git-History ist nicht “die Wahrheit”

Verzerrungen durch:

- Rebase & Squash-Merges
- Pairing (nur eine Person commitet)
- Bot-Commits (Dependabot, etc.)
- Vendored/Generated Code
- Repo-Rewrites (z.B. git-filter-branch)

Häufige Fehl-Interpretationen:

- Commits/Tag = Produktivität ✗
- Lines of Code = Qualität ✗
- Ownership = Domain-Wissen ✗
- Coupling = Kommunikation ✗
- Sentiment = Team-Kultur ✗

⚠️ Technische Limits:

- Burndown kann OOM laufen (große Repos mit vielen Branches)
- YAML-Parsing sehr teuer → PB verwenden
- Babelfish ist abandoned (im Roadmap als “to remove”)
- Couples braucht Tensorflow (speicherhungrig)

Was fehlt? Roadmap

Fehlende Metriken

Was Hercules NICHT misst:



Bus-Factor als Zahl

- Ownership ist da, aber keine Aggregat-KPI



Onboarding-KPIs

- Time-to-first-meaningful-change
- Breadth-of-files in ersten 30 Tagen



Code-Review-Metriken

- Zykluszeit, Review-Latenz
- Rework nach Review
- (Braucht GitHub/GitLab API)

Vorschläge für neue Metriken:



Bus-Factor@80% (Repo/Subsystem)

- Kleinste k mit $\text{Sum}(\text{Top-}k) \geq 80\%$
- **Aufwand:** Niedrig (Ownership-Daten da)



Ownership-Konzentrationsindex

- Gini oder HHI über Zeit
- **Aufwand:** Niedrig-Mittel



Onboarding-Ramp

- Zeit bis X Commits/Changes
- Cohort-Plots
- **Aufwand:** Mittel

Kombinierte Risk-Scores

- Hotspot × Coupling × Ownership
- Als fertige Metrik

Hotspot-Risikoscore

- $\log(\text{size}) \times \text{churn} \times \text{coupling} \times \text{concentration}$
- **Aufwand:** Mittel–Hoch

Knowledge Diffusion

- Unique editors per file
- **Aufwand:** Mittel

Zusammenfassung

Was wir gelernt haben

Die wichtigsten Punkte:

✓ **Hercules macht Git-Historie messbar**

- Burndown, Ownership, Coupling, Hotspots, Zeitreihen

✓ **Pipeline-Architektur**

- Composable, effizient, erweiterbar

✓ **Teamdynamik als “Spur im Code”**

- Ownership-Entwicklung, Onboarding, Zusammenarbeitsmuster

✓ **Grenzen verstehen**

- Git ≠ Wahrheit; Metriken sind Proxies; Kontext nötig

Was du mitnehmen kannst:

Für Forschung/Analyse:

- Hercules als Werkzeug für Code-Archäologie
- Vergangenheit sichtbar machen, Risiken erkennen

Für die Lehre:

- Kritisches Denken über Metriken
- “Measure twice, interpret carefully”

Für die Praxis:

- Bus-Factor, Hotspots, Onboarding messbar machen
- Aber: Metriken allein reichen nicht – Kontext ist King!

Vielen Dank!

Fragen? Diskussion?

 Fork: github.com/MeKo-Christian/hercules

Basierend auf src-d/hercules (ursprünglich source{d})

Appendix

GIT VERGISST NICHTS.

DIG THE DIFF. PARSE DEN PAST.

Ich wollte nur kurz was fixen...
→ berüchte letzte Worte.

DIE GRUNDIDEE

DEIN CODE HAT GEDÄCHTNIS.
DU NICHT.

- Commit = Momentaufnahme (unterschrieben)
- Hash = Fingerabdruck (Beweisstück-ID)
- Diff = Narbe. (zeigt, was passiert ist)

**QUITTUNGEN.
FÜR ALLES.**

DIE REPO-DETEKTEI.

WENIG DRAMA. VIEL PROTOKOLL.

- 1) git diff – Fakten statt Gefühl.
- 2) git log – die Zeitleiste deiner Entscheidungen.
- 3) git blame – „Wer hat's angefasst?“ (Spiegel, kein Messer. Mestens).

PUSH – force →
ZEITPARADOXIEN & CHAOS MÖGLICH.

*Auf eigene Verantwortung, Held.

METRIKEN!

AUS GESCHICHTE WERDEN ZAHLEN.

- Churn: Was wird ständig umgeschrieben? (Zappel-Dateien)
- Hotspots: Wo brennt's immer wieder? (Risiko-Zonen)
- Ownership: Wer hält hier allein den Laden? (Bus-Faktor)

MESSE DIE NARBEN.

DIE 3 GESETZE DES REPO-GEDÄCHTNISSES:

1. Alles wird erinnert – wenn du commitest.
2. Alles ist messbar – wenn du's sauber schreibst.
3. Alles ist rekonstruierbar – wenn du Geschichte nicht zerschredderst.

KOMMIT KLEIN. SCHREIB MENSCHLICHE MESSAGES.

Hercules & Labours: Quick Start

Schritt 1: Metrics mit Hercules extrahieren

```
# Aggregierung mit burndown, couples und dev Informationen  
hercules --burndown --couples --dev --pb <repo-url> > repo.pb
```

Schritt 2: Charts mit Labours generieren

```
# Generierung von Charts über Python mit matplotlib  
labours -m burndown-project -f pb repo.pb -o burndown.png
```

Verfügbare labours-Modes:

burndown-project, burndown-file, burndown-person, code-ownership, devs, devs-efforts, couples, old-vs-new, languages, sentiment

Weitere Informationen & Workflows

Im Hercules Repository:

- Vollständige Dokumentation im README.md
- Build: `just hercules`
- Alle Analysis-Optionen: `hercules --help`
- Labours installieren: `just install-labours`